

FOCUS ASTU CP HUB

Master System Design Document

Version 3.0

Focus ASTU Competitive Programming Community

Adama Science and Technology University

Document Status: Final Draft | May 2026

Table of Contents

Table of Contents.....	2
1. Executive Summary.....	5
2. System Overview & Goals.....	5
2.1 Core Features (MVP).....	5
2.2 Design Constraints.....	6
3. Technology Stack.....	6
4. High-Level Architecture.....	7
4.1 System Topology.....	7
4.2 Request Flow Summary.....	8
5. Database Design.....	8
5.1 Database Enums.....	8
5.2 Table Definitions.....	8
Table: squads.....	8
Table: users.....	9
Table: invitations.....	9
Table: system_settings.....	10
Table: user_roles_history.....	10
Table: problems.....	10
Table: submissions.....	11
Table: contests.....	11
Table: contest_standings.....	12
Table: squad_tracks.....	12
Table: squad_track_topics.....	12
Table: topic_problems.....	13
Table: editorials.....	13
Table: announcements.....	13
5.3 Row Level Security Policies.....	14
6. Authentication & Authorization.....	15
6.1 Core Principles.....	15
6.2 Registration & Invitation Flow.....	15
Path A — Public Signup (when enabled by Super Admin).....	15
Path B — Admin Invitation (default / primary path).....	15
6.3 Profile Completion Flow.....	15
6.4 Role Hierarchy & Permission Matrix.....	16
6.5 Route Protection Strategy (Frontend).....	16
7. Backend Design (Go — Clean Architecture).....	17
7.1 Directory Structure.....	17
7.2 Core Use Cases.....	17
UC-01: Problem Submission.....	17

UC-02: Codeforces Contest Sync.....	18
UC-03: Squad Track Management.....	18
UC-04: Daily Verse.....	18
7.3 REST API Endpoint Reference.....	18
Public Endpoints (No Auth Required).....	19
Authenticated Endpoints (Any Active, Non-Banned User).....	19
Squad Lead Endpoints.....	19
Admin & Super Admin Endpoints.....	20
7.4 Background Services.....	20
Daily Verse Cache.....	20
7.5 Deployment (Docker).....	20
8. Frontend Design (React + Vite).....	21
8.1 Architecture & State Management.....	21
8.2 Directory Structure.....	21
8.3 Page & Feature Inventory.....	22
8.4 Browser Extension Integration.....	23
9. Landing Page & Navigation Model.....	23
9.1 Landing Page.....	23
9.1.1 Landing Page Navbar.....	23
9.1.2 Landing Page Content.....	23
9.1.3 What the Landing Page Does NOT Show.....	24
9.2 Announcements Page (/announcements).....	24
New Backend Endpoint Required.....	24
Updated RLS / Endpoint Behaviour for Announcements.....	24
9.3 Navigation Model.....	24
9.3.1 Two Distinct Shells.....	24
9.3.2 Sidebar Navigation Items.....	25
9.3.3 Dashboard Page Content.....	25
10. Browser Extension Design (Core MVP).....	25
10.1 Overview & Supported Platforms.....	25
10.2 Extension Architecture.....	26
10.3 Authentication Handshake.....	26
10.4 Submission Capture Flow.....	26
11. Phase 2 — Future Features.....	27
12. Implementation Roadmap.....	27
Phase 1 — Foundation (Weeks 1–2).....	27
Phase 2 — Backend Core (Weeks 2–3).....	28
Phase 3 — Frontend Scaffolding (Weeks 3–4).....	28
Phase 4 — Core Features (Weeks 4–6).....	28
Phase 5 — Contest & Squad Features (Weeks 6–8).....	29
Phase 6 — Browser Extension (Weeks 7–9, parallel).....	29

Phase 7 — Deployment & Hardening (Weeks 9–10)..... 29

13. Non-Functional Requirements..... 30

1. Executive Summary

Focus ASTU CP Hub is a private, invite-only competitive programming portal built for the Focus ASTU tech community at Adama Science and Technology University. The platform is modelled after elite CP training hubs (e.g., A2SV) and provides a centralized space for tracking problem-solving progress, running internal contests, managing squad-based curriculum, and building a culture of competitive accountability.

The system is organized around the concept of Squads — cohorts of members analogous to "generations" at A2SV — each led by Squad Leaders who design and deliver their squad's algorithmic curriculum. Admins manage the overall platform, while a Super Admin controls global system settings.

The platform consists of three interconnected components:

- A Go backend implementing Clean Architecture, exposed as a REST API.
- A React frontend (Vite + TypeScript) consuming both the Go API and Supabase directly.
- A browser extension (Chrome/Firefox) that automatically captures accepted submissions from LeetCode, Codeforces, and AtCoder and posts them to the portal.

2. System Overview & Goals

2.1 Core Features (MVP)

Feature	Description
Invite-Only Registration	Users join via an admin-generated invitation link or when public signup is explicitly enabled by a Super Admin.
Squad-Based Organization	Members belong to numbered Squads (e.g., 1st Squad, 2nd Squad). Each squad has its own lead, curriculum, and announcements.
Problem Tracking	Master list of problems from LeetCode, Codeforces, and AtCoder. Members log solutions; the platform tracks counts and streaks.
Code Viewing	Submitted code is stored and viewable by all members with syntax highlighting. No code execution is performed.
Contest Sync	Admins sync Codeforces contests by contest ID. The system fetches standings, maps participants to portal users, and tracks upsolves.
Editorials	Any member can write a Markdown editorial for any problem.
Squad Curriculum Tracks	Squad Leads design curriculum trees (Track → Topics → Problems) for their squad.
Announcements	Global announcements (from Admins) and squad-scoped announcements (from Squad Leads).
Browser Extension	Auto-captures accepted solutions from LeetCode, Codeforces, and AtCoder and submits them to the portal silently.
Daily Verse	A Bible verse fetched from a public API, cached server-side, and displayed on the Landing Page.

Feature	Description
Admin Dashboard	User management, role assignment, squad assignment, ban management, and contest sync.

2.2 Design Constraints

- Deployment target is a budget VPS (1–2 GB RAM). All design decisions must respect this constraint.
- No code execution engine. The platform stores and displays code only.
- Authentication is entirely delegated to Supabase Auth. No custom password management.
- Frontend design (colors, fonts, animations) is defined separately and is excluded from this document.

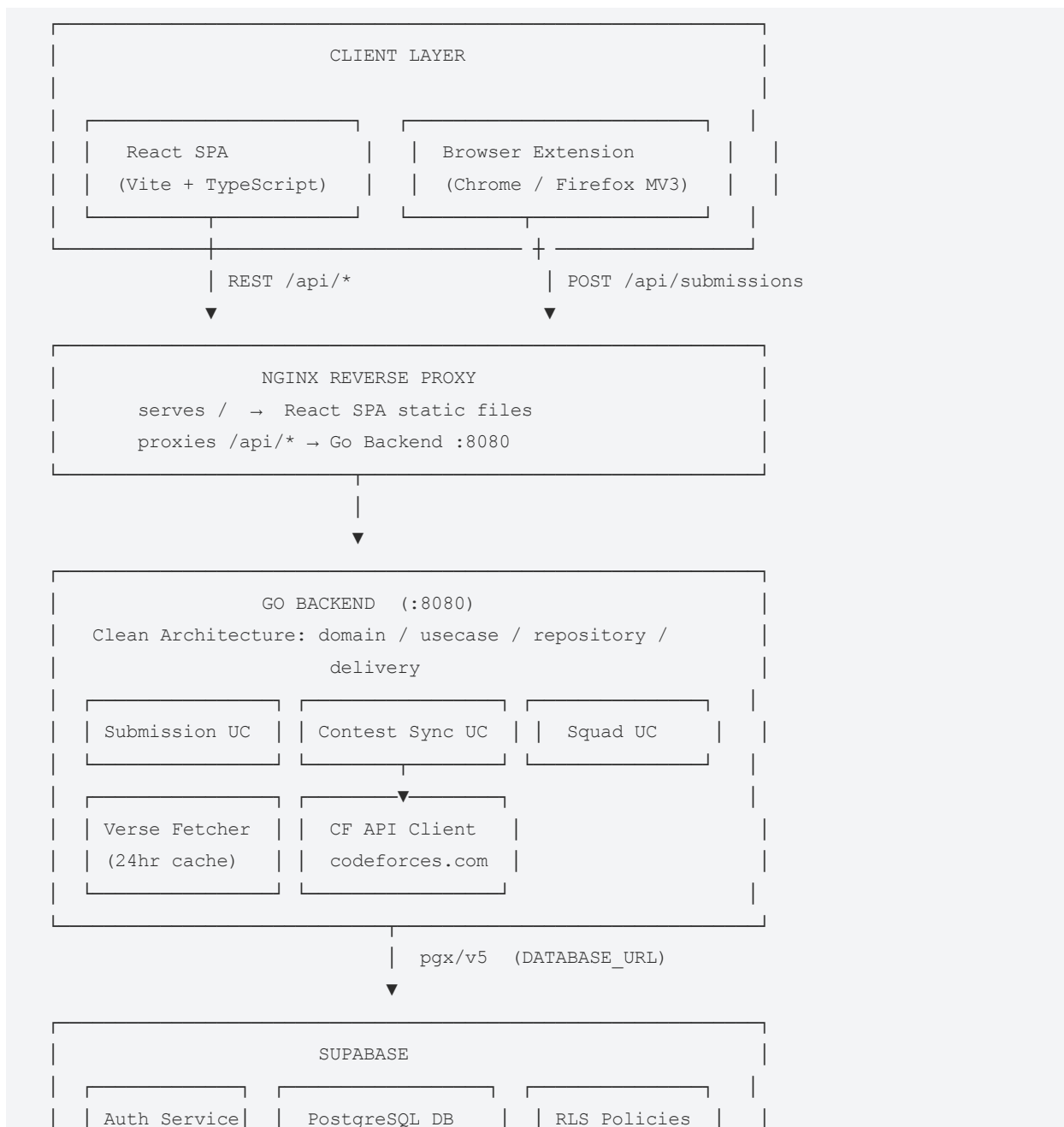
3. Technology Stack

Layer	Technology	Rationale
Backend Language	Go (Golang) 1.21+	~15 MB container image, high concurrency via goroutines, ideal for budget VPS hosting.
Backend Framework	Echo v4	Lightweight HTTP router with good middleware support for JWT and CORS.
Database	PostgreSQL via Supabase	Managed PostgreSQL with built-in Auth, Row Level Security, and real-time capabilities.
Database Client (Go)	pgx/v5 (pgxpool)	High-performance PostgreSQL driver with connection pooling.
Authentication	Supabase Auth	Handles magic links, invite flows, JWT issuance, and session management.
Frontend Framework	React 18 + Vite + TypeScript	Fast HMR in development, optimized production builds, strong typing.
Frontend Routing	React Router DOM v6	SPA navigation with nested layout routes and route guards.
Frontend Data Fetching	TanStack Query	Server-state caching, background refetching, and stale-while-revalidate strategy.
Frontend State	Zustand	Minimal global state for UI toggles. TanStack Query handles all server state.
Supabase Client	@supabase/supabase-js	Auth state management and direct DB reads where no Go business logic is required.
Containerization	Docker + Docker Compose	Reproducible builds, simple VPS deployment. Multi-stage builds keep images minimal.
Reverse Proxy	Nginx (in Docker)	Serves the built React SPA and proxies /api/* to the Go container.
Browser Extension	Manifest V3 (Chrome/Firefox)	Modern extension standard with background service workers and content scripts.

Layer	Technology	Rationale
External APIs	Codeforces API, Bible Verse API	Free, public APIs requiring no authentication.

4. High-Level Architecture

4.1 System Topology



4.2 Request Flow Summary

Scenario	Flow
User logs in	React → Supabase Auth (magic link) → Supabase returns JWT → stored in React context
User views problem list	React → Supabase JS client directly (RLS-protected SELECT on problems) — bypasses Go to save compute
Extension submits code	Extension → Nginx → Go API (JWT validated) → Postgres → streak/count updated
Squad Lead syncs CF contest	React → Nginx → Go API (Squad Lead JWT + squad_id check) → Go calls Codeforces API → results saved to Postgres
User reads dashboard	React → Go API GET /api/verse (cached) + Supabase direct for user stats

5. Database Design

5.1 Database Enums

All enums are defined as PostgreSQL ENUM types for type safety at the database level.

```
CREATE TYPE role_type AS ENUM (
  'SUPER_ADMIN', 'ADMIN', 'SQUAD_LEAD', 'SQUAD_MEMBER', 'COMMUNITY'
);

CREATE TYPE platform_type AS ENUM (
  'LEETCODE', 'CODEFORCES', 'ATCODER', 'OTHER'
);

CREATE TYPE announcement_scope AS ENUM (
  'GLOBAL', 'SQUAD'
);
```

5.2 Table Definitions

Table: squads

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Primary key

Column	Type	Constraints	Description
name	TEXT	NOT NULL, UNIQUE	e.g. "1st Squad", "2nd Squad"
created_at	TIMESTAMPTZ	DEFAULT now()	Creation timestamp

Table: users

Maps 1:1 to auth.users. Stores all application-level profile data. Created via a Supabase Auth trigger on first login.

Column	Type	Constraints	Description
id	UUID	PK, FK → auth.users.id ON DELETE CASCADE	Mirrors Supabase Auth user ID
email	TEXT	NOT NULL, UNIQUE	User email (synced from auth)
full_name	TEXT	NOT NULL	Display name
bio	TEXT	NULLABLE	Short personal bio
telegram_handle	TEXT	NOT NULL (required at completion)	Telegram username (no @)
linkedin_url	TEXT	NULLABLE	LinkedIn profile URL
leetcode_handle	TEXT	NULLABLE	LeetCode username
codeforces_handle	TEXT	NULLABLE	Used for CF contest matching
atcoder_handle	TEXT	NULLABLE	AtCoder handle
squad_id	UUID	NULLABLE, FK → squads.id ON DELETE SET NULL	NULL = COMMUNITY role
role	role_type	NOT NULL, DEFAULT 'COMMUNITY'	User's current role
is_banned	BOOLEAN	NOT NULL, DEFAULT false	Banned users get 403 on all writes
is_active	BOOLEAN	NOT NULL, DEFAULT false	False until profile completion
problem_count	INTEGER	NOT NULL, DEFAULT 0	Denormalized total unique solves
daily_streak	INTEGER	NOT NULL, DEFAULT 0	Consecutive days with a submission
last_submission_date	DATE	NULLABLE	Used to compute streak continuity
created_at	TIMESTAMPTZ	DEFAULT now()	Account creation time

Table: invitations

Tracks admin-generated invite tokens. Enforces that only the intended email can consume each token.

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Primary key
email	TEXT	NOT NULL	The email this invitation is locked to
token	TEXT	NOT NULL, UNIQUE	Cryptographically random 32-byte hex token
created_by	UUID	FK → users.id	The admin who generated the invite
expires_at	TIMESTAMPTZ	NOT NULL	Validity window (e.g. 72 hours from creation)
used_at	TIMESTAMPTZ	NULLABLE	Set on consumption; NULL means still valid
created_at	TIMESTAMPTZ	DEFAULT now()	Creation timestamp

Table: system_settings

A key-value store for global platform configuration. Managed exclusively by Super Admins.
Seed: INSERT INTO system_settings (key, value) VALUES ('signup_open', 'false');

Column	Type	Constraints	Description
key	TEXT	PRIMARY KEY	Setting name, e.g. signup_open
value	TEXT	NOT NULL	String value, e.g. "false"
updated_at	TIMESTAMPTZ	DEFAULT now()	Last modified timestamp

Table: user_roles_history

Immutable log of role and squad assignments over time. Powers dynamic badge generation (e.g. "2nd Squad Lead").

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Primary key
user_id	UUID	NOT NULL, FK → users.id ON DELETE CASCADE	The user whose role changed
squad_id	UUID	NULLABLE, FK → squads.id	Squad context at time of role assignment
role	role_type	NOT NULL	The role assigned
assigned_at	TIMESTAMPTZ	DEFAULT now()	When this role was assigned

Table: problems

Master catalog of all problems across all platforms. Problems are created automatically when an extension submission references an unknown external_id.

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Internal problem ID
name	TEXT	NOT NULL	Human-readable problem name
platform	platform_type	NOT NULL	Source platform
external_id	TEXT	NOT NULL	Platform-specific identifier (e.g. 1A, two-sum)
external_link	TEXT	NOT NULL	Direct URL to the problem page
tags	TEXT[]	DEFAULT '{}'	Algorithmic tags, e.g. {dp, math, greedy}
created_at	TIMESTAMPTZ	DEFAULT now()	When first added to the portal
— UNIQUE —		(platform, external_id)	Prevents duplicate problem entries

Table: submissions

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Submission ID
user_id	UUID	NOT NULL, FK → users.id ON DELETE CASCADE	The submitting user
problem_id	UUID	NOT NULL, FK → problems.id ON DELETE RESTRICT	The problem solved
language	TEXT	NOT NULL	e.g. C++, Python, Java
code	TEXT	NOT NULL	Raw accepted source code
is_contest	BOOLEAN	NOT NULL, DEFAULT false	True if submitted during a tracked contest
contest_id	UUID	NULLABLE, FK → contests.id	Associated contest (when is_contest = true)
source	TEXT	NOT NULL, DEFAULT 'manual'	'manual' or 'extension'
submitted_at	TIMESTAMPTZ	DEFAULT now()	Submission timestamp

Table: contests

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Internal contest ID
name	TEXT	NOT NULL	Contest display name
platform	platform_type	NOT NULL, DEFAULT 'CODEFORCES'	Source platform

Column	Type	Constraints	Description
external_id	TEXT	NOT NULL, UNIQUE	Codeforces contest ID, e.g. 1932
held_at	TIMESTAMPTZ	NOT NULL	When the contest took place
synced_at	TIMESTAMPTZ	DEFAULT now()	Last fetch from Codeforces
created_at	TIMESTAMPTZ	DEFAULT now()	Row creation timestamp

Table: contest_standings

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Primary key
contest_id	UUID	NOT NULL, FK → contests.id ON DELETE CASCADE	The contest
user_id	UUID	NOT NULL, FK → users.id ON DELETE CASCADE	The participant
rank	INTEGER	NOT NULL	Final contest rank
old_rating	INTEGER	NULLABLE	Codeforces rating before contest
new_rating	INTEGER	NULLABLE	Codeforces rating after contest
problems_solved	INTEGER	NOT NULL, DEFAULT 0	Count solved during contest
upsolved_count	INTEGER	NOT NULL, DEFAULT 0	Count solved after contest
— UNIQUE —		(contest_id, user_id)	One standing per user per contest

Table: squad_tracks

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Primary key
squad_id	UUID	NOT NULL, FK → squads.id ON DELETE CASCADE	Owning squad
title	TEXT	NOT NULL	Track name, e.g. "Python Foundations"
created_at	TIMESTAMPTZ	DEFAULT now()	Creation timestamp

Table: squad_track_topics

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Primary key

Column	Type	Constraints	Description
track_id	UUID	NOT NULL, FK → squad_tracks.id ON DELETE CASCADE	Parent track
title	TEXT	NOT NULL	Topic name, e.g. "Hashmaps", "DP Basics"
order_index	INTEGER	NOT NULL, DEFAULT 0	Display order within a track
created_at	TIMESTAMPTZ	DEFAULT now()	Creation timestamp

Table: topic_problems

Column	Type	Constraints	Description
topic_id	UUID	NOT NULL, FK → squad_track_topics.id ON DELETE CASCADE	Parent topic
problem_id	UUID	NOT NULL, FK → problems.id ON DELETE CASCADE	Linked problem
added_at	TIMESTAMPTZ	DEFAULT now()	When the problem was added
— PK —		(topic_id, problem_id)	Composite primary key

Table: editorials

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Primary key
problem_id	UUID	NOT NULL, FK → problems.id ON DELETE CASCADE	Problem this covers
user_id	UUID	NOT NULL, FK → users.id ON DELETE CASCADE	Author
content_md	TEXT	NOT NULL	Full editorial in Markdown format
created_at	TIMESTAMPTZ	DEFAULT now()	Submission timestamp

Table: announcements

Column	Type	Constraints	Description
id	UUID	PK, DEFAULT gen_random_uuid()	Primary key
author_id	UUID	NOT NULL, FK → users.id	The user who created the announcement
squad_id	UUID	NULLABLE, FK → squads.id	NULL = global; non-null = squad-scoped

Column	Type	Constraints	Description
title	TEXT	NOT NULL	Announcement headline
body	TEXT	NOT NULL	Body content (Markdown supported)
created_at	TIMESTAMPTZ	DEFAULT now()	Publish timestamp

5.3 Row Level Security Policies

RLS is enabled on every table. The Go backend connects using the service_role key, which bypasses RLS for trusted server operations. The React frontend uses the user JWT and is subject to all policies below.

Table	Role	SELECT	INSERT	UPDATE	DELETE
users	Any authenticated	All rows	—	Own row only	—
users	ADMIN / SUPER_ADMIN	All rows	—	Any row	—
problems	Any authenticated	All rows	— (Go only)	—	—
submissions	Any authenticated	All rows	Own user_id, not banned	—	—
contests	Any authenticated	All rows	— (Go only)	—	—
contest_standings	Any authenticated	All rows	— (Go only)	—	—
editorials	Any authenticated	All rows	Own user_id	Own row	Own row
squad_tracks	SQUAD_LEAD	All rows	Own squad only	Own squad only	Own squad only
squad_track_topics	SQUAD_LEAD	All rows	Own squad only	Own squad only	Own squad only
topic_problems	SQUAD_LEAD	All rows	Own squad only	—	Own squad only
announcements	Any authenticated	Global + own squad	— (Go only)	—	—
system_settings	SUPER_ADMIN	All rows	All rows	All rows	—
invitations	ADMIN / SUPER_ADMIN	All rows	All rows	All rows	—

6. Authentication & Authorization

6.1 Core Principles

- All authentication state is owned by Supabase Auth. No custom session management is implemented.
- `auth.users` is the source of truth for identity. `public.users` stores extended profile metadata, kept in sync via a database trigger on `auth.users` insert.
- The Go backend validates requests by verifying the Supabase-issued JWT using the `SUPABASE_JWT_SECRET` environment variable (HMAC-SHA256).
- The sub claim in the JWT is used as the `user_id` for all write operations. The client never sends a `user_id` in request body for protected endpoints.

6.2 Registration & Invitation Flow

Path A — Public Signup (when enabled by Super Admin)

1. Frontend checks GET `/api/system/signup-status`.
2. If `signup_open = true`, user fills email on `/signup`.
3. Frontend calls `supabase.auth.signUp({ email, password })` or magic link.
4. Supabase Auth trigger creates a stub row in `public.users` with `role = 'COMMUNITY'` and `is_active = false`.
5. User is redirected to `/complete-profile`.

Path B — Admin Invitation (default / primary path)

6. Admin navigates to Admin Dashboard → Invitations tab.
7. Admin enters target email. The Go backend generates a cryptographically random 32-byte hex token, stores it in `invitations` with a 72-hour expiry, and sends an email containing a link to `/invite?token=abc123`.
8. User visits `/invite?token=abc123`.
9. Frontend validates token via GET `/api/invite/validate?token=abc123`. Returns the locked email or an error.
10. If valid, the signup form is pre-filled and locked to that email.
11. On successful `supabase.auth.signUp()`, the Go backend marks `invitations.used_at = now()`.
12. User is redirected to `/complete-profile`.

6.3 Profile Completion Flow

A new user account is considered INACTIVE until mandatory profile fields are submitted. An inactive user is locked to the `/complete-profile` route exclusively.

Mandatory fields at profile completion:

- Full Name
- Telegram Handle

- Codeforces Handle (required — used for contest matching and standings sync)
- LeetCode Handle (optional)
- AtCoder Handle (optional)

On successful form submission, the Go backend sets `is_active = true` and `role = 'COMMUNITY'` in `public.users`. The user is then redirected to `/dashboard`.

6.4 Role Hierarchy & Permission Matrix

Roles in ascending privilege order: **COMMUNITY < SQUAD_MEMBER < SQUAD_LEAD < ADMIN < SUPER_ADMIN**

Permission	COM	MBR	LEAD	ADMIN	S_ADMIN
View problems & submissions	✓	✓	✓	✓	✓
Submit code (manual or extension)	✓	✓	✓	✓	✓
Write editorials	✓	✓	✓	✓	✓
View global announcements	✓	✓	✓	✓	✓
View squad announcements (own squad)	✗	✓	✓	✓	✓
Post squad announcements (own squad)	✗	✗	✓	✓	✓
Post global announcements	✗	✗	✗	✓	✓
Manage squad curriculum (own squad)	✗	✗	✓	✓	✓
Add COMMUNITY users to own squad	✗	✗	✓	✓	✓
Generate invitations	✗	✗	✗	✓	✓
Ban / unban users	✗	✗	✗	✓	✓
Assign roles (up to SQUAD_LEAD)	✗	✗	✗	✓	✓
Assign roles (up to ADMIN)	✗	✗	✗	✗	✓
Sync Codeforces contests (own squad)	✗	✗	✓	✓	✓
Toggle signup_open	✗	✗	✗	✗	✓

COM = COMMUNITY | MBR = SQUAD_MEMBER | LEAD = SQUAD_LEAD | S_ADMIN = SUPER_ADMIN

6.5 Route Protection Strategy (Frontend)

User State	Attempted Route	Outcome
Unauthenticated	Any protected route	Redirect → /login
Authenticated, <code>is_active = false</code>	Any except /complete-profile	Redirect → /complete-profile
Authenticated, <code>is_active = true</code>	/complete-profile	Redirect → /dashboard
Authenticated, active, wrong role	/admin/* without ADMIN role	Redirect → /dashboard
Authenticated, active, correct role	Any allowed route	Render normally

7. Backend Design (Go — Clean Architecture)

7.1 Directory Structure

```

backend/
├── cmd/server/main.go           # Entry point: wire deps, start server
├── internal/
│   ├── domain/
│   │   ├── models.go           # Entities: User, Problem, Submission, etc.
│   │   ├── enums.go           # Go constants mirroring DB enums
│   │   └── repository.go       # Repository interfaces (contracts)
│   └── usecase/
│       ├── submission_uc.go    # Submit problem, upsert, update stats
│       ├── contest_uc.go      # Sync CF contest, list, get standings
│       ├── squad_uc.go        # Manage tracks, topics, problems
│       ├── user_uc.go         # Profile completion, user queries
│       ├── admin_uc.go        # Role management, bans, invitations
│       ├── editorial_uc.go     # Create & list editorials
│       └── verse_uc.go         # Fetch & cache daily Bible verse
│   └── repository/postgres/
│       ├── db.go              # pgxpool connection setup
│       ├── user_repo.go
│       ├── problem_repo.go
│       ├── submission_repo.go
│       ├── contest_repo.go
│       ├── squad_repo.go
│       ├── editorial_repo.go
│       └── announcement_repo.go
│   └── delivery/http/
│       ├── server.go          # Echo router setup, route registration
│       ├── middleware.go      # JWT auth + role guard middleware
│       ├── context.go         # UserIDFromContext, etc.
│       └── [feature]_handler.go # One handler file per feature
│   └── pkg/codeforces/client.go # HTTP client for Codeforces API
│   └── config/config.go       # Env var loading
├── Dockerfile
├── .env.example
└── go.mod

```

7.2 Core Use Cases

UC-01: Problem Submission

Triggered by: Extension POST or manual POST to `/api/submissions`

13. Validate JWT. Extract `user_id` from token claims.
14. Fetch user from DB. Verify `is_banned = false` and `is_active = true`. Return 403 if either fails.
15. Look up problems by (platform, `external_id`). If not found, UPSERT a new problem using name and link from the request payload.
16. Insert a new row into submissions.
17. Invoke `UpdateUserStats`: if (`user_id`, `problem_id`) pair is new, increment `problem_count`. Compare `last_submission_date`: same day = no change; yesterday = increment streak; older/null = reset streak to 1. Update `last_submission_date` = today.
18. Return 201 Created with submission ID and updated user stats.

UC-02: Codeforces Contest Sync

Triggered by: Squad Lead POST to `/api/squads/:squadID/contests/sync`, or Admin POST to `/api/admin/contests/sync`

19. Verify caller has `SQUAD_LEAD` (and their `squad_id` matches the URL) or `ADMIN` / `SUPER_ADMIN` role.
20. Call Codeforces API: `contest.standings?contestId={id}&showUnofficial=false` with a 15-second timeout.
21. Parse response. Extract contest metadata and participant rows.
22. Upsert the contests row (keyed on `external_id`).
23. For each CF participant, look up matching users row by `codeforces_handle`. Skip unmatched participants silently.
24. Upsert `contest_standings` rows for matched users.
25. Identify problems unsolved during contest for matched users — populates the upsolve view.
26. Return sync summary: `{ matched_users: N, standings_saved: M, contest_id: "..." }`.

UC-03: Squad Track Management

Triggered by: Squad Lead via `/api/squads/{id}/tracks`

27. JWT middleware extracts user. Role guard verifies `role == SQUAD_LEAD`.
28. Validate caller's `squad_id` matches the `squad_id` in the URL parameter. Return 403 if mismatched.
29. Execute the requested operation (create track / add topic / assign problem).

UC-04: Daily Verse

Triggered by: GET `/api/verse`

30. Check in-memory cache. If a verse was fetched within the last 24 hours, return it immediately.
31. Otherwise, call the external Bible Verse API. On failure, return the last known cached verse.
32. Update the cache with the new verse and timestamp.
33. Return `{ text: "...", reference: "..." }`.

7.3 REST API Endpoint Reference

Public Endpoints (No Auth Required)

Method	Path	Description
GET	/api/healthz	Health check. Returns {"status":"ok"}.
GET	/api/verse	Returns the cached daily Bible verse.
GET	/api/system/signup-status	Returns whether public signup is enabled.
GET	/api/invite/validate	Validates an invite token. Query param: ?token=
GET	/api/announcements/public	Returns the N most recent global announcements. Used by the landing page.
GET	/api/stats/public	Returns total member count, total problems solved, total contests. Used by landing page.

Authenticated Endpoints (Any Active, Non-Banned User)

Method	Path	Description
GET	/api/users/:userID	Get user profile with computed badges.
PUT	/api/users/me	Update own profile (bio, handles, telegram).
POST	/api/users/me/complete-profile	Submit mandatory profile fields → sets is_active = true.
GET	/api/problems	List all problems. Optional: ?platform=&tag=
POST	/api/submissions	Create a submission (manual or extension).
GET	/api/submissions/:submissionID	Get a single submission with code and metadata.
GET	/api/users/:userID/submissions	Get paginated submissions for a user.
GET	/api/contests	List all synced contests.
GET	/api/contests/:contestID/standings	Get standings (includes upsolve info).
GET	/api/editorials	List editorials. Query: ?problem_id=
POST	/api/editorials	Create an editorial (Markdown body).
GET	/api/squads/:squadID/tracks	Get full curriculum tree for a squad.
GET	/api/announcements	List announcements visible to the caller.

Squad Lead Endpoints

Method	Path	Description
POST	/api/squads/:squadID/tracks	Create a new curriculum track.
POST	/api/tracks/:trackID/topics	Add a topic to a track.
POST	/api/topics/:topicID/problems	Assign a problem to a topic.
POST	/api/announcements	Post a squad-scoped announcement.

Method	Path	Description
POST	/api/squads/:squadID/contests/sync	Sync a Codeforces contest for own squad. Body: {"contest_id":"1932"}

Admin & Super Admin Endpoints

Method	Path	Description
GET	/api/admin/users	List all users with full metadata.
PUT	/api/admin/users/:userID/role	Update a user's role. Body: {"role":"SQUAD_LEAD"}
PUT	/api/admin/users/:userID/squad	Assign user to a squad. Body: {"squad_id":"..."}
PUT	/api/admin/users/:userID/ban	Ban or unban a user. Body: {"is_banned":true}
POST	/api/admin/invitations	Generate an invitation. Body: {"email":"..."}
POST	/api/admin/contests/sync	Sync any Codeforces contest (cross-squad). Body: {"contest_id":"1932"}
POST	/api/admin/announcements	Post a global announcement.
PUT	/api/admin/system/signup	(Super Admin only) Toggle {"open":true}

7.4 Background Services

Daily Verse Cache

A simple in-memory struct guarded by a sync.RWMutex, holding the last fetched verse text, reference, and fetch timestamp. All requests to GET /api/verse read from this cache. A background goroutine refreshes it every 24 hours. On startup, the cache is populated immediately before the HTTP server starts accepting requests, ensuring the first request is never a cache miss.

7.5 Deployment (Docker)

```
# backend/Dockerfile
FROM golang:1.21-alpine AS builder
WORKDIR /app
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 GOOS=linux go build -ldflags="-s -w" -o server cmd/server/main.go

FROM alpine:3.19
RUN apk add --no-cache ca-certificates
WORKDIR /app
```

```
COPY --from=builder /app/server .
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s \
  CMD wget -qO- http://localhost:8080/api/healthz || exit 1
EXPOSE 8080
CMD ["/server"]
```

```
# docker-compose.yml
version: "3.9"
services:
  backend:
    build: ./backend
    env_file: ./backend/.env
    restart: unless-stopped
  frontend:
    build: ./frontend
    ports:
      - "3000:80"
    depends_on:
      backend:
        condition: service_healthy
    restart: unless-stopped
```

8. Frontend Design (React + Vite)

8.1 Architecture & State Management

The frontend follows a feature-first directory structure. All server state (data fetching, caching, mutations) is handled by TanStack Query. Zustand is used only for ephemeral UI state (sidebar open/closed, active tab). Authentication state is managed via a dedicated AuthContext wrapping Supabase Auth's `onAuthStateChange`.

For simple read operations that do not require Go business logic (e.g., listing problems, reading user profiles), the frontend may query Supabase directly using the JS client, subject to RLS. This reduces load on the Go backend.

8.2 Directory Structure

```
frontend/
├─ src/
│   ├─ app/router.tsx           # All routes, ProtectedRoute wrappers
│   ├─ components/
│   │   ├─ layout/             # AppShell, Sidebar, Header
│   │   └─ ui/                 # Badge, StatCard, CodeViewer, MarkdownRenderer
│   └─ features/
```

└─ landing/	# LandingPage (public), LandingNavbar
└─ auth/	# LoginPage, SignupPage, InvitePage,
	# CompleteProfilePage, ProtectedRoute
└─ dashboard/	# DashboardPage (verse + stats + activity)
└─ problems/	# ProblemsPage, SubmissionViewPage,
	# EditorialPage, ProblemFilters
└─ contests/	# ContestsPage, ContestDetailPage
└─ profile/	# ProfilePage (badges, handles, streaks)
└─ squad/	# SquadPage (curriculum tree accordion)
└─ announcements/	# AnnouncementsPage
└─ admin/	# AdminPage (lazy-loaded)
└─ hooks/	# useAuth, useUserProfile
└─ lib/	# supabase.ts, api.ts (Axios + JWT)
└─ store/	# uiStore.ts (Zustand)
└─ main.tsx	# QueryClientProvider, RouterProvider
└─ Dockerfile	
└─ nginx.conf	
└─ package.json	

8.3 Page & Feature Inventory

Route	Access	Key Functionality
/	Public	Landing page: hero, daily verse, global announcements feed, community stats. Public navbar with Announcements and Dashboard links.
/login	Public	Email + magic link via Supabase Auth
/signup	Public (if open)	Standard email signup via Supabase Auth
/invite	Public	Token validation, locked email signup
/complete-profile	Auth + inactive	Mandatory profile form; sets is_active = true
/dashboard	Auth + active	Daily verse card, personal stats, recent activity
/problems	Auth + active	Filterable problem list with platform badges and submission accordion
/submissions/:id	Auth + active	Syntax-highlighted code viewer with copy button
/problems/:id/editorials	Auth + active	Read and write Markdown editorials
/contests	Auth + active	List of synced contests with date and participant count
/contests/:id	Auth + active	Standings table with upsolve toggle
/profile/:userID	Auth + active	Badges, platform handles, streaks, recent submissions
/squad	SQUAD_MEMBER +	Curriculum tree: Track → Topics → Problems
/announcements	Public + Auth	Public: global announcements only. Authenticated: global + squad-scoped. Squad Leads and Admins see a Post button.

Route	Access	Key Functionality
/admin	ADMIN+	User management, role/squad controls, ban toggle, invitations, contest sync

8.4 Browser Extension Integration

The portal generates a personal API Key for each user on the Settings page (/settings/extension). This key is passed as a Bearer token by the extension. The API key is stored as a SHA-256 hash in public.users and validated by the Go backend on each extension request.

This approach avoids requiring the extension to manage Supabase JWTs directly and allows key revocation without affecting the user's auth session.

9. Landing Page & Navigation Model

9.1 Landing Page

The Landing Page is the first page any visitor sees — authenticated or not. It is fully public and requires no login. Its purpose is to give an overview of the community, surface global announcements, and display the daily Bible verse.

9.1.1 Landing Page Navbar

The navbar is always visible on the landing page and contains:

- Focus ASTU CP Hub logo / wordmark (left side)
- Announcements — navigates to /announcements (public view, shows only global non-protected announcements)
- Dashboard button — navigates to /dashboard. If the user is not logged in, redirects to /login first.
- Login / Sign Up button — shown only when the user is not authenticated.

9.1.2 Landing Page Content

Section	Content	Auth Required
Hero Banner	Community name, tagline, and a call-to-action button ("Join" or "Go to Dashboard" depending on auth state).	No
Daily Verse Card	Bible verse of the day fetched from the cached Go endpoint GET /api/verse. Displayed prominently with the reference.	No
Global Announcements Feed	The 5 most recent global announcements (squad_id = NULL). Each card shows title, a short excerpt of the body, and the date. Links to the full /announcements page.	No
Community Stats (optional)	Simple counters: total members, total problems solved, total contests run. Fetched from a new public endpoint GET /api/stats/public.	No

9.1.3 What the Landing Page Does NOT Show

- Squad-scoped announcements — those are protected and only visible after login.
- Any user-specific data (profile, submissions, streaks).
- The Daily Verse card does not require authentication — the `/api/verse` endpoint is already public.

9.2 Announcements Page (/announcements)

The Announcements page is accessible from the landing page navbar and from the authenticated sidebar. Its content changes depending on the viewer's auth state:

Viewer State	Content Shown
Unauthenticated	Only global announcements (squad_id = NULL). Read-only. No create button.
Authenticated, COMMUNITY	Global announcements only. Read-only.
Authenticated, SQUAD_MEMBER / SQUAD_LEAD	Global announcements + squad-scoped announcements for their squad. Squad Leads also see a "Post Announcement" button.
ADMIN / SUPER_ADMIN	All announcements. "Post Global Announcement" button visible.

The same `/announcements` route serves both the public landing page context and the authenticated app context. The frontend uses the auth state to decide which announcements to request and whether to show the create button.

New Backend Endpoint Required

Method	Path	Auth	Description
GET	<code>/api/announcements/public</code>	None	Returns the N most recent global announcements. Used by the landing page.
GET	<code>/api/stats/public</code>	None	Returns total member count, total problems solved, total contests. Used by landing page stats.

Updated RLS / Endpoint Behaviour for Announcements

The existing GET `/api/announcements` endpoint (authenticated) already filters by global + own squad. The new GET `/api/announcements/public` endpoint is strictly public and returns only global announcements with no auth check.

9.3 Navigation Model

The application uses a sidebar-based navigation model for all authenticated pages. There are no tabs within the dashboard — each section is a full page reached via the sidebar.

9.3.1 Two Distinct Shells

Shell	Routes	Description
Landing Shell	/, /announcements (public)	Minimal navbar only. No sidebar. Public-facing. No auth required.
App Shell	/dashboard, /problems, /contests, /profile, /squad, /announcements, /admin, /settings	Full sidebar + header. Auth + active required. Each item is a distinct page.

9.3.2 Sidebar Navigation Items

Sidebar Item	Route	Visible To
Dashboard	/dashboard	All authenticated active users
Problems	/problems	All authenticated active users
Contests	/contests	All authenticated active users
My Squad	/squad	SQUAD_MEMBER, SQUAD_LEAD
Announcements	/announcements	All authenticated active users
Profile	/profile/:myUserID	All authenticated active users
Settings	/settings/extension	All authenticated active users
Admin	/admin	ADMIN, SUPER_ADMIN only

9.3.3 Dashboard Page Content

The dashboard (/dashboard) is a personal home screen, not a tab container. It shows:

- Personal stats: total problems solved, current streak, squad name and role badge.
- Recent submissions: the last 5 submissions by the logged-in user with problem name, language, and time.
- Recent global announcements: a small preview of the 3 latest announcements (links to /announcements).
- Quick links: buttons to Problems, Contests, and Squad pages.

The Daily Verse has moved to the Landing Page. It is not shown on the authenticated dashboard.

10. Browser Extension Design (Core MVP)

10.1 Overview & Supported Platforms

The browser extension is a Manifest V3 extension targeting Chrome (primary) and Firefox. It detects when a user successfully submits a problem on a supported judge and automatically forwards the accepted code to the Focus ASTU portal backend. AtCoder and any OTHER platform problems must be submitted manually through the portal.

Platform	Auto-Capture	Detection Method	Data Captured
LeetCode	✓ Auto	Intercept fetch response to /submissions/detail/* checking for "statusDisplay": "Accepted"	Problem slug, language, code, runtime
Codeforces	✓ Auto	Observe DOM for the "Accepted" verdict on the submission result page	Problem ID (e.g. 1A), language, code
AtCoder	Manual only	— Extension not supported. User submits code manually via the portal.	—
OTHER	Manual only	— User pastes problem link and code directly in the portal.	—

10.2 Extension Architecture

```

extension/
├─ manifest.json                # MV3 manifest
├─ background/
│   └─ service_worker.js       # Receives messages, calls portal API
├─ content_scripts/
│   ├── leetcode.js            # Injected on leetcode.com
│   └─ codeforces.js           # Injected on codeforces.com
├─ popup/
│   ├── popup.html             # Connection status + API key entry
│   └─ popup.js
└─ icons/

```

10.3 Authentication Handshake

34. User installs the extension.
35. User opens the extension popup and visits portal /settings/extension to copy their personal API key.
36. User pastes the API key into the extension popup. The extension stores it in chrome.storage.sync.
37. The extension sends a test request to GET /api/healthz with the API key header to confirm connectivity.
38. Popup displays a green "Connected" badge or an error message.

Security: The Go backend stores only the SHA-256 hash of the API key. The raw key is shown to the user exactly once on generation. On each request, the backend hashes the provided key and compares it to the stored hash.

10.4 Submission Capture Flow

39. Content Script detects an accepted submission on the judge's page (via DOM observation or fetch interception).

40. Content Script sends a message to the Service Worker via `chrome.runtime.sendMessage` containing: platform (LEETCODE | CODEFORCES), `external_id`, `problem_name`, `external_link`, language, and code.
41. Service Worker retrieves the API key from `chrome.storage.sync`.
42. Service Worker sends `POST /api/submissions` with Authorization: Bearer <api_key>.
43. On success (201): Service Worker shows a browser notification confirming the submission was saved.
44. On failure: submission is queued in `chrome.storage.local` for retry (max 3 retries with exponential backoff).

Important: The extension sends code only for ACCEPTED verdicts. Wrong Answer, TLE, Runtime Error, and Compilation Error submissions are silently ignored.

11. Phase 2 — Future Features

Feature	Description	Rationale for Deferral
GitHub Sync	After a submission is saved, a Go background worker formats and pushes the code file to the user's connected GitHub repository via the GitHub Contents API. File path: <code>{platform}/{problem_name}/{language}.{ext}</code> . OAuth flow for account connection.	Requires OAuth implementation and a persistent worker queue. Not essential for core tracking.
Internal Contest Rating	A localized ELO-based rating system computed from contest standings, separate from Codeforces' official rating. Updates on each synced contest.	Algorithm design and edge cases require careful design before implementation.
Global Leaderboard	Rankings by problem count, streak, and internal contest rating.	Derivable from existing data once a rating system is in place.
AtCoder & LeetCode Contest Sync	Extend contest syncing beyond Codeforces.	Each platform has a different API. Codeforces is prioritized due to community usage.
Firefox Extension	Port the Chrome MV3 extension to Firefox's WebExtensions API.	Chrome is the primary target. Firefox port is straightforward once Chrome is stable.
Mobile App	React Native or PWA wrapper for native mobile experience.	The web app will be mobile-responsive. A native app is out of scope for MVP.

12. Implementation Roadmap

Phase 1 — Foundation (Weeks 1–2)

#	Task	Owner
1.1	Initialize Git repository with monorepo structure (/backend, /frontend, /extension).	Backend
1.2	Create Supabase project. Retrieve SUPABASE_URL, SUPABASE_ANON_KEY, SUPABASE_JWT_SECRET, service_role key.	Backend
1.3	Write and apply all SQL migrations: enums, tables, RLS policies, auth.users trigger.	Backend
1.4	Seed initial data: 1st Squad row, system_settings.signup_open = false.	Backend
1.5	Initialize Go module. Scaffold Clean Architecture directory structure.	Backend
1.6	Implement pgxpool connection, config loading from env vars, and health check endpoint.	Backend

Phase 2 — Backend Core (Weeks 2–3)

#	Task	Owner
2.1	Define all domain models and repository interfaces.	Backend
2.2	Implement all PostgreSQL repository implementations.	Backend
2.3	Implement JWT auth middleware (validate Supabase JWT, extract user_id).	Backend
2.4	Implement role guard middleware.	Backend
2.5	Implement UC-01: Problem Submission use case + POST /api/submissions.	Backend
2.6	Implement API Key generation, hashing, and validation for extension auth.	Backend
2.7	Implement user profile endpoints (GET, PUT, complete-profile).	Backend
2.8	Implement Daily Verse use case and endpoint.	Backend

Phase 3 — Frontend Scaffolding (Weeks 3–4)

#	Task	Owner
3.1	Initialize Vite + React + TypeScript project. Install all dependencies.	Frontend
3.2	Configure React Router with all routes, Landing Shell, App Shell, and ProtectedRoute.	Frontend
3.3	Implement AuthContext using Supabase onAuthStateChanged.	Frontend
3.4	Build Landing Page: hero, daily verse card, global announcements feed, public navbar.	Frontend
3.5	Build Login, Signup, Invite, and Complete Profile pages.	Frontend
3.6	Configure Axios instance with automatic JWT attachment and TanStack Query client.	Frontend
3.7	Build App Shell (sidebar + header) with navigation items.	Frontend

Phase 4 — Core Features (Weeks 4–6)

#	Task	Owner
4.1	Dashboard Page: daily verse card + personal stats (problem count, streak, squad).	Frontend

#	Task	Owner
4.2	Problems Page: filterable table with platform badges and expandable submission accordion.	Frontend + Backend
4.3	Submission View Page: syntax-highlighted code with copy-to-clipboard button.	Frontend
4.4	Editorial Page: Markdown reader and writer.	Frontend + Backend
4.5	Profile Page: badges, handles, streak display, recent submissions list.	Frontend + Backend

Phase 5 — Contest & Squad Features (Weeks 6–8)

#	Task	Owner
5.1	UC-02: Codeforces contest sync use case + squad lead endpoint (own squad) + admin endpoint (any squad).	Backend
5.2	Contests Page + Contest Detail Page (standings + upsolve toggle).	Frontend
5.3	UC-03: Squad Track Management use case + endpoints.	Backend
5.4	Squad Page: curriculum tree accordion (Track → Topic → Problem).	Frontend
5.5	Announcements Page + creation forms (admin/squad lead).	Frontend + Backend
5.6	Admin Dashboard: user table, role/squad controls, ban toggle, invitations, contest sync.	Frontend + Backend

Phase 6 — Browser Extension (Weeks 7–9, parallel)

#	Task	Owner
6.1	Scaffold MV3 extension with manifest, content scripts, service worker, and popup.	Extension
6.2	LeetCode content script: detect accepted verdict, extract code.	Extension
6.3	Codeforces content script: detect AC verdict, extract code.	Extension
6.4	Service worker: receive messages, call portal API, retry queue with backoff.	Extension
6.5	Popup UI: API key entry, connection status, last submission indicator.	Extension
6.6	Portal Settings page for API key generation and extension download link.	Frontend + Backend

Phase 7 — Deployment & Hardening (Weeks 9–10)

#	Task	Owner
7.1	Write multi-stage Dockerfiles for backend and frontend.	DevOps
7.2	Write docker-compose.yml with health checks, restart policies, env_file references.	DevOps

#	Task	Owner
7.3	Configure Nginx for SPA fallback, /api proxy, and gzip compression.	DevOps
7.4	End-to-end testing: registration, extension submission, contest sync, role escalation.	All
7.5	Deploy to VPS. Configure DNS and HTTPS (Let's Encrypt).	DevOps

13. Non-Functional Requirements

Category	Requirement	Implementation Approach
Performance	API responses under 200ms for non-sync operations on a 1GB VPS.	pgxpool connection reuse, TanStack Query caching on frontend, in-memory verse cache.
Security	No client-provided user_id on write endpoints. API keys stored as SHA-256 hashes. Supabase RLS as second enforcement layer.	JWT middleware, hashed API keys, RLS policies, banned-user check on every write endpoint.
Reliability	Graceful degradation if Codeforces API or Bible Verse API is unreachable.	Timeouts on all external HTTP calls. Fallback to cached verse. CF sync errors return descriptive failure, not 500.
Resource Efficiency	Total Docker memory usage under 300MB on a 1GB VPS.	Multi-stage Docker builds (~15MB Go binary, ~30MB Nginx + static files). No Node.js in production.
Maintainability	Backend business logic must be testable without a real database or HTTP server.	Clean Architecture with repository interfaces. Unit tests target use case layer with mock repositories.
Scalability	Architecture allows future migration to public signup, Kubernetes, or additional integrations without major rewrites.	Docker-first design. Single config toggle for public signup. Clean separation of concerns throughout.
Data Integrity	Duplicate (user, problem) pairs are de-duplicated for problem count purposes.	UpdateUserStats checks for existing (user_id, problem_id) before incrementing problem_count.

— End of Document —

Focus ASTU CP Hub · Master System Design v3.0